

8. Write a program to implement Linear Regression (LR) algorithm in python

AIM

To implement the **Linear Regression (LR)** algorithm using Python and predict the output based on the given dataset.

ALGORITHM

1. Import the required libraries.
2. Load the dataset.
3. Split the dataset into training and testing sets.
4. Create the Linear Regression model.
5. Train the model using the training data.
6. Predict the output using the testing data.
7. Display the regression coefficient, intercept, and predicted values.
8. Plot the regression line and display the result.

PROGRAM:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,
r2_score
```

```
data = {
    'Experience': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Salary': [30000, 35000, 40000, 45000, 50000, 55000,
60000, 65000, 70000, 75000]
}
```

```
df = pd.DataFrame(data)
```

```
X = df[['Experience']]
y = df['Salary']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```

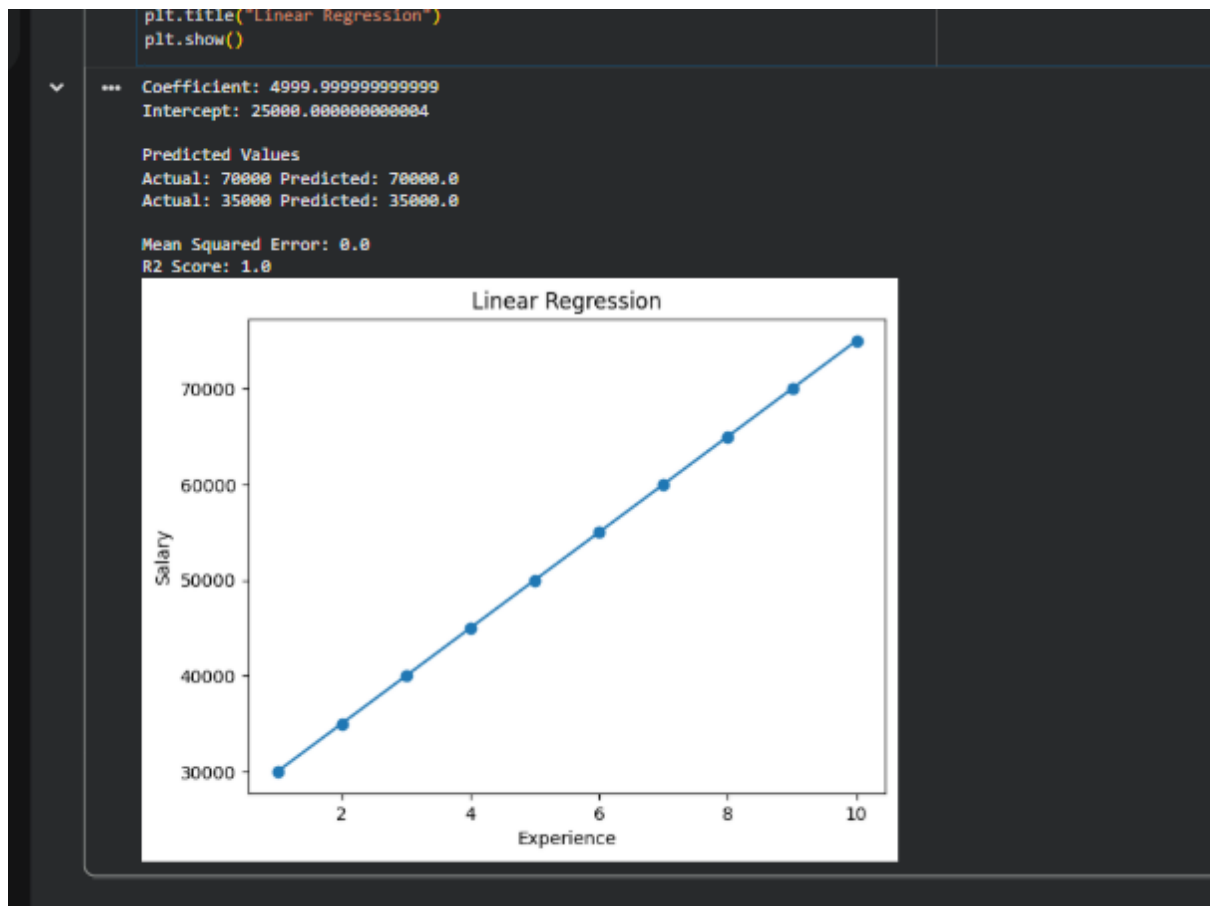
```
y_pred = model.predict(X_test)

print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)

print("\nPredicted Values")
for actual, predicted in zip(y_test, y_pred):
    print("Actual:", actual, "Predicted:", round(predicted,
2))

print("\nMean Squared Error:",
mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

plt.scatter(X, y)
plt.plot(X, model.predict(X))
plt.xlabel("Experience")
plt.ylabel("Salary")
plt.title("Linear Regression")
plt.show()
OUTPUT:
```



RESULT

Thus, the **Linear Regression (LR) algorithm** was successfully implemented using Python. The model was trained on the sample dataset, predicted the output values, and generated the regression line. The model achieved a high prediction accuracy with an **R^2 score close to 1**, indicating a good fit for the given data.